

TECHNICAL REPORT: WEB APPLICATION SECURITY INVESTIGATION



UNIVERSIDAD

Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 14 - Web Application Security

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-05-05

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Introduction

1.1 Objective

The primary objective of this laboratory is to master the identification, interception, and exploitation of common web application vulnerabilities as defined by the OWASP Top 10. We aim to utilize proxy interception tools (Burp Suite) to manipulate HTTP traffic, bypass medium-level security filters in the Damn Vulnerable Web App (DVWA), and understand the underlying insecure coding practices to propose robust mitigations.

1.2 Discussion: The Web Security Perspective

Perspective: Andersson David Sánchez Méndez

"Web security fundamentally challenges the assumption of trust. Because HTTP is stateless, the security of an entire application relies heavily on how it manages sessions and sanitizes inputs. If an attacker can intercept a session token or bypass input validation, the application's perimeter effectively ceases to exist."

Perspective: Cristian Santiago Pedraza Rodríguez

"Vulnerability research requires seeing every input field, HTTP header, and URL parameter as a potential entry point. By mastering traffic interception tools like Burp Suite, we move from being passive consumers of web interfaces to active auditors, uncovering the hidden logic flaws that lead to critical compromises."



2. Core Principles of Web Application Security

The investigation is governed by four fundamental pillars to ensure applications are resilient against modern web attacks:

Principle	Description	Implementation Strategy
Input Validation	Never trust data sent by the client.	Parameterized queries, Output encoding (HTML entity encoding), Whitelisting.
Session Management	Securely maintaining state across HTTP requests.	Strong, unpredictable session IDs, HttpOnly, and Secure cookie flags.
Access Control	Ensuring users only access permitted resources.	Server-side validation of object ownership and CSRF Tokens.
Traffic Visibility	Monitoring and analyzing client-server interaction.	Utilizing Interception Proxies (e.g., Burp Suite) for deep packet inspection.



3. Web Architecture and Protocol Analysis

We distinguish between the structural components of the web and how data flows statelessly between them:

3.1 The Client-Server Model

- 1. **Frontend (Client):** User interface executing HTML, CSS, and JavaScript. Vulnerable to client-side attacks like DOM-based XSS.
- 2. **Backend (Server):** Business logic and persistent storage (Database). Vulnerable to server-side attacks like SQL Injection and Local File Inclusion (LFI).

3.2 HTTP State and Cookies

- ◆ **Statelessness:** Each HTTP request is independent. The server retains no memory of previous connections.
- ◆ **Session Tracking:** Applications use cookies (`Cookie: session_id= ...`) to simulate state. These must be protected with attributes like `HttpOnly` and `SameSite` to prevent unauthorized extraction or cross-site request forgery.



4. OWASP Top 10 Fundamentals

Understanding the mechanics behind critical web vulnerabilities is essential for exploitation and defense:

Vulnerability	Mechanism	Attack Goal
SQL Injection (SQLi)	Malicious SQL syntax alters backend query logic.	Bypass authentication, extract or modify database records.
Cross-Site Scripting (XSS)	Execution of malicious JavaScript in the victim's browser.	Session hijacking, unauthorized actions, credential theft.
Broken Access Control	Failure to enforce authorization boundaries.	Accessing other users' sensitive data or administrative functions.
Local File Inclusion (LFI)	Manipulating input to include local system files.	Reading sensitive files (<code>/etc/passwd</code>) or escalating to RCE.



5. PHASE 1: ENVIRONMENT SETUP & TRAFFIC INTERCEPTION

This phase focuses on establishing the testing environment using Kali Linux and deploying the **Damn Vulnerable Web App (DVWA)**. Furthermore, we configured Burp Suite to act as a Man-in-the-Middle (MitM) proxy to intercept HTTP traffic.

- ◆ **Tools Used:** Kali Linux terminal, Firefox web browser, Burp Suite, Wireshark.
- ◆ **Commands Used:** `sudo bash -c "$(curl --fail --show-error --silent --location https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"`

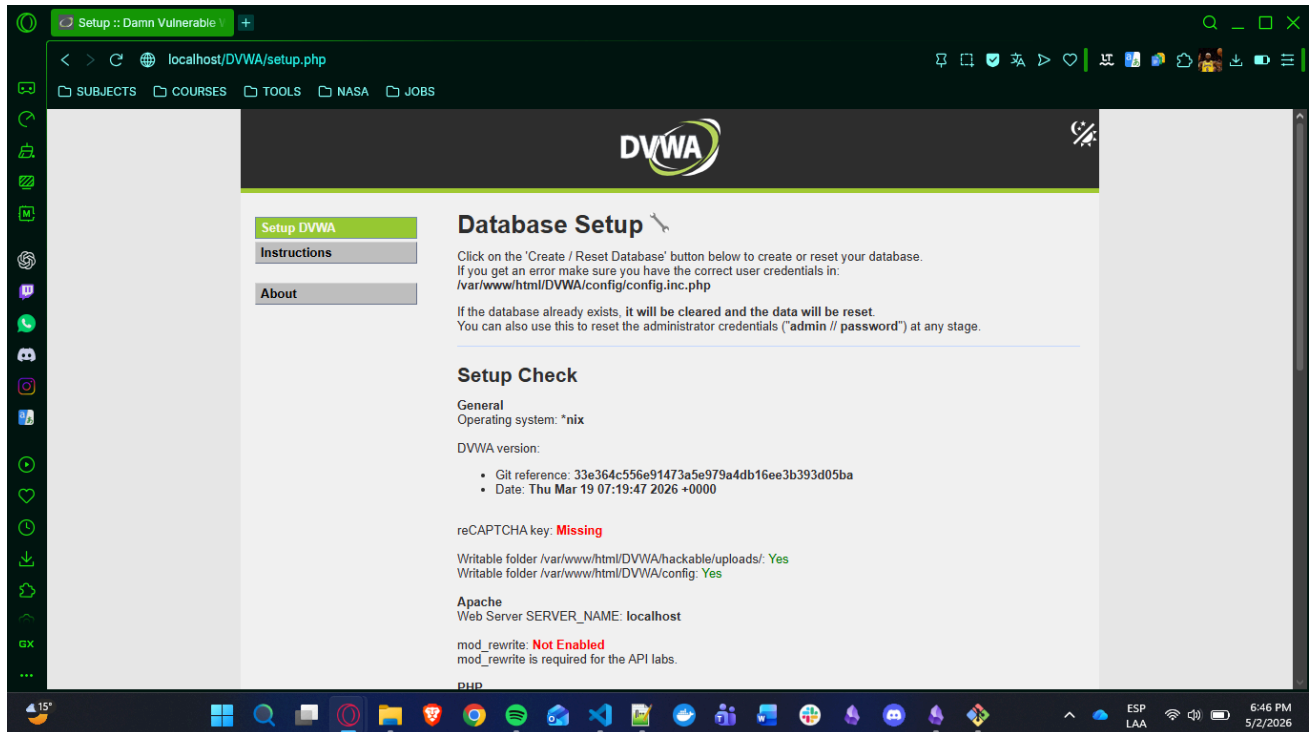
Evidence Annex: Phase 1 (Environment)

◆ DVWA Initialization:

```
andres@kali:~/DVWA$ sudo bash -c "$(curl --fail --show-error --silent --location https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"
Unpacking php-gd (2:8.3+93ubuntu2) ...
Setting up php8.3-gd (8.3.6-0ubuntu0.24.04.8) ...
Creating config file /etc/php/8.3/mods-available/gd.ini with new version
Setting up php-gd (2:8.3+93ubuntu2) ...
Processing triggers for libapache2-mod-php8.3 (8.3.6-0ubuntu0.24.04.8) ...
Processing triggers for php8.3-cli (8.3.6-0ubuntu0.24.04.8) ...
libapache2-mod-php is not installed. Installing it now ...
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libgl1-mesa-dri libglapi-mesa libllvm17t64
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  libapache2-mod-php
0 upgraded, 1 newly installed, 0 to remove and 7 not upgraded.
Need to get 4324 B of archives.
After this operation, 15.4 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libapache2-mod-php all 2:8.3+93ubuntu2 [4324 B]
Fetched 4324 B in 2s (1966 B/s)
Selecting previously unselected package libapache2-mod-php.
(Reading database ... 80691 files and directories currently installed.)
Preparing to unpack .../libapache2-mod-php_2:8.3+93ubuntu2_all.deb ...
Unpacking libapache2-mod-php (2:8.3+93ubuntu2) ...
Setting up libapache2-mod-php (2:8.3+93ubuntu2) ...
git is installed!
Downloading DVWA from GitHub ...
Cloning into '/var/www/html/DVWA' ...
remote: Enumerating objects: 333, done.
remote: Counting objects: 100% (333/333), done.
remote: Compressing objects: 100% (279/279), done.
remote: Total 333 (delta 41), reused 265 (delta 24), pack-reused 0 (from 0)
Receiving objects: 100% (333/333), 973.62 KiB | 2.68 MiB/s, done.
Resolving deltas: 100% (41/41), done.
MariaDB service is already enabled.
MariaDB service is already running.
Default credentials:
Username: root
Password: [No password just hit Enter]
Enter SQL user:

Key concepts
```

◆ Login Successful:



The screenshot shows the 'Setup DVWA' page in a web browser. The browser's address bar displays 'localhost/DVWA/setup.php'. The page has a dark header with the DVWA logo. On the left, there is a sidebar with links: 'Setup DVWA' (highlighted), 'Instructions', and 'About'. The main content area is titled 'Database Setup' and contains the following text:

Click on the 'Create / Reset Database' button below to create or reset your database. If you get an error make sure you have the correct user credentials in: `/var/www/html/DVWA/config/config.inc.php`

If the database already exists, it will be cleared and the data will be reset. You can also use this to reset the administrator credentials ("admin // password") at any stage.

Setup Check

General
Operating system: *nix

DVWA version:

- Git reference: 33e364c556e91473a5e979a4db16ee3b393d05ba
- Date: Thu Mar 19 07:19:47 2026 +0000

reCAPTCHA key: **Missing**

Writable folder `/var/www/html/DVWA/hackable/uploads/`: **Yes**
Writable folder `/var/www/html/DVWA/config/`: **Yes**

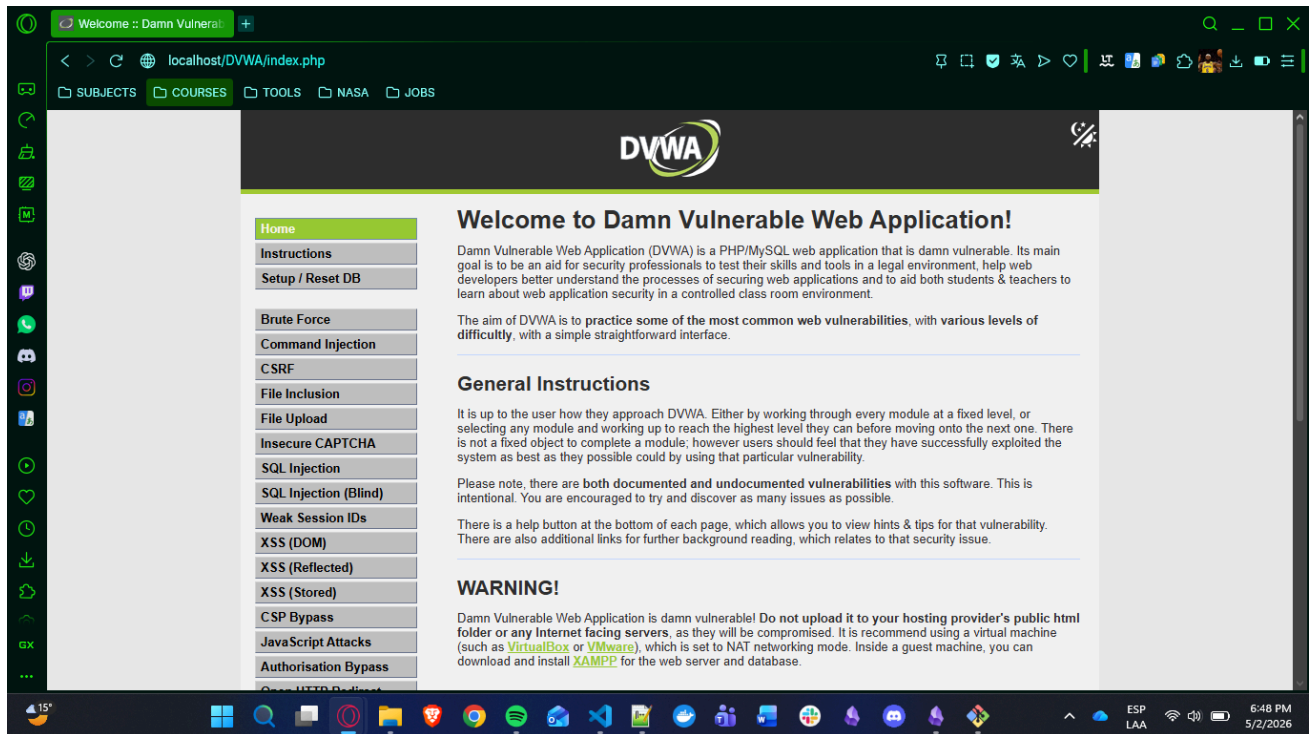
Apache
Web Server SERVER_NAME: localhost

mod_rewrite: **Not Enabled**
mod_rewrite is required for the API labs.

DHDP

The Windows taskbar at the bottom shows the time as 6:46 PM on 5/2/2026.

◆ Database Restore:



The screenshot shows the 'Welcome to Damn Vulnerable Web Application!' page in a web browser. The browser's address bar displays 'localhost/DVWA/index.php'. The page has a dark header with the DVWA logo. On the left, there is a sidebar with links: 'Home' (highlighted), 'Instructions', 'Setup / Reset DB', 'Brute Force', 'Command Injection', 'CSRF', 'File Inclusion', 'File Upload', 'Insecure CAPTCHA', 'SQL Injection', 'SQL Injection (Blind)', 'Weak Session IDs', 'XSS (DOM)', 'XSS (Reflected)', 'XSS (Stored)', 'CSP Bypass', 'JavaScript Attacks', and 'Authorisation Bypass'. The main content area contains the following text:

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module, however users should feel that they have successfully exploited the system as best as they possibly could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any Internet facing servers, as they will be compromised. It is recommended using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

The Windows taskbar at the bottom shows the time as 6:48 PM on 5/2/2026.

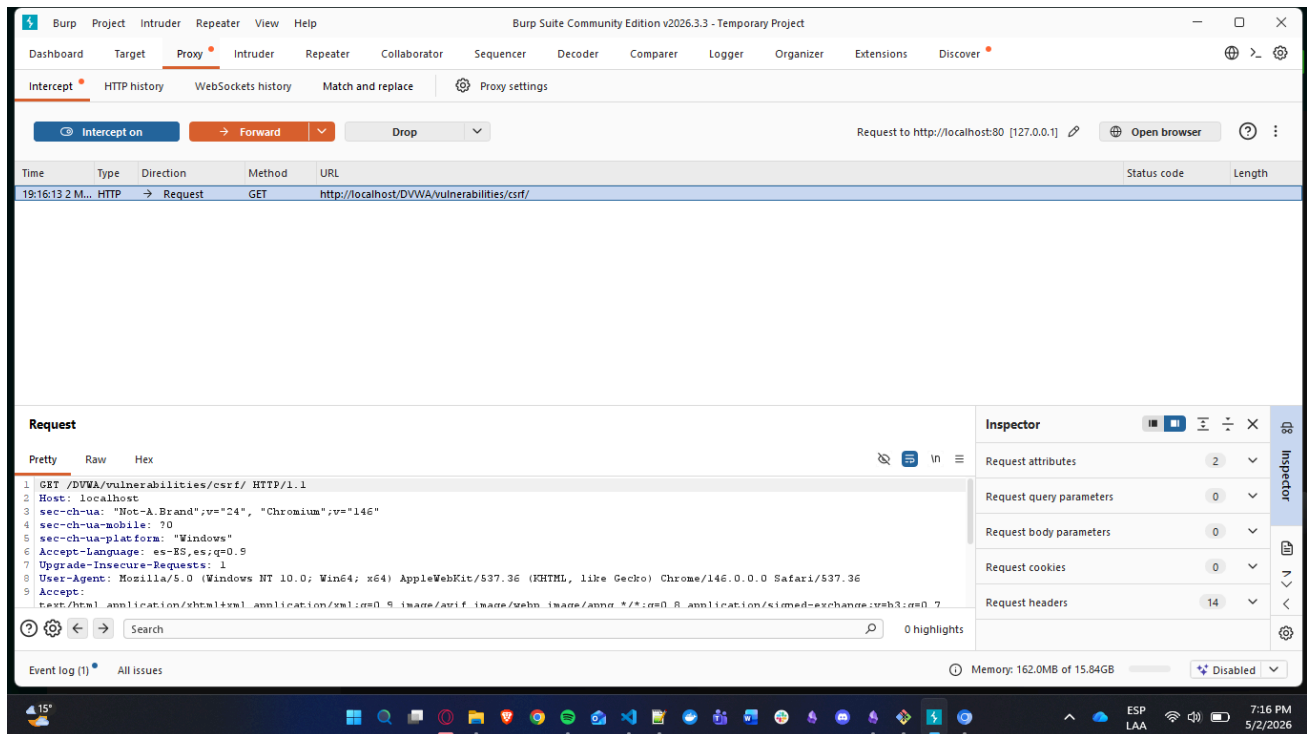
◆ Security Level:

The screenshot shows the DVWA Security Level page in a web browser. The browser's address bar displays 'localhost/DVWA/security.php'. On the left, a sidebar menu lists various security challenges: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, and API. The main content area is titled 'DVWA Security' and 'Security Level'. It states that the current security level is 'medium'. Below this, a list of four levels is provided: 1. Low (completely vulnerable), 2. Medium (teaches bad security practices), 3. High (extension of medium difficulty), and 4. Impossible (secure against all vulnerabilities). A dropdown menu is set to 'Medium' with a 'Submit' button. Under 'Additional Tools', there is a link to 'View Broken Access Control Logs'. At the bottom, a message states 'Security level set to medium'.

◆ Network Capture:

The screenshot shows a Wireshark network capture of an HTTP GET request. The packet list on the left shows 'Frame 438: 336 bytes on wire (2688 bits), 336 bytes captured (2688 bits) on interface \Device\NPF_{B038E2D2-5499-4361-8D33-03146E9B5980}, id 0'. The packet details pane shows the following structure: Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data in hexadecimal and ASCII. The ASCII part of the packet bytes pane shows the following text: 'GET /msdownload/update/v3/static/trusted/en/pinrulesstl.cab?id=0f3681076d18 HTTP/1.1'. The packet status bar at the bottom shows 'No. 438 - Time: 41.648623 - Source: 192.168.1.3 - Destination: 23.218.213.6 - Protocol: HTTP - Length: 336 - Info: GET /msdownload/update/v3/static/trusted/en/pinrulesstl.cab?id=0f3681076d18 HTTP/1.1'. The packet bytes pane also shows the following text: '0 H: p...f...E...B: @...w...: PKK, ...P...PM: GE T /msdownload/update/v3/static/trusted/en/pinrulesstl.cab?id=0f3681076d18 HTTP/1.1 Connection: Keep-Alive Accept: */* If-Modified-Since: Fri, 02 Jun 2017 17:39:05 GMT If-None-Match: "804 24021c7d'.

◆ Proxy Capture:



6. TECHNICAL DISCUSSION: INSECURE CODING PRACTICES

? Before starting each exploit, click “View Source” in DVWA to read the server-side code. What specific insecure coding practice enables each vulnerability?

- 1. SQL Injection / Blind SQLi:** The code utilizes `mysql_real_escape_string()`, which escapes quotes. However, because the user ID is an integer, it is placed into the query *without* surrounding quotes (e.g., `WHERE user_id = $id;`). This allows attackers to inject logical operators directly without needing to escape out of a string context.
- 2. XSS (Reflected/Stored):** The developer relies on `$name = str_replace('<script>', '', $_GET['name']);`. The insecure practice is using a simple, case-sensitive blacklist. Attackers can bypass this by simply changing the capitalization (e.g., `<Script>`).
- 3. XSS (DOM):** The server uses `stripes($default, "<script") !== false` to check the URL parameter. The insecure practice is attempting to mitigate client-side DOM execution using a weak server-side blacklist that ignores alternative HTML tags like `<img>`.
- 4. File Inclusion:** The code attempts to block directory traversal using `$file = str_replace(array("../", "..\\"), "", $file);`. The insecure practice is

using a non-recursive string replacement.

5. **File Upload:** The application relies on `$uploaded_type = "image/jpeg"`. The insecure practice is trusting the `Content-Type` HTTP header sent by the client, which is trivial to spoof.
6. **CSRF:** The server implements an `HTTP_REFERER` check using `stripes()`. The insecure practice is checking if the server's name simply *exists anywhere* within the referer string, rather than ensuring it is the exact domain origin.

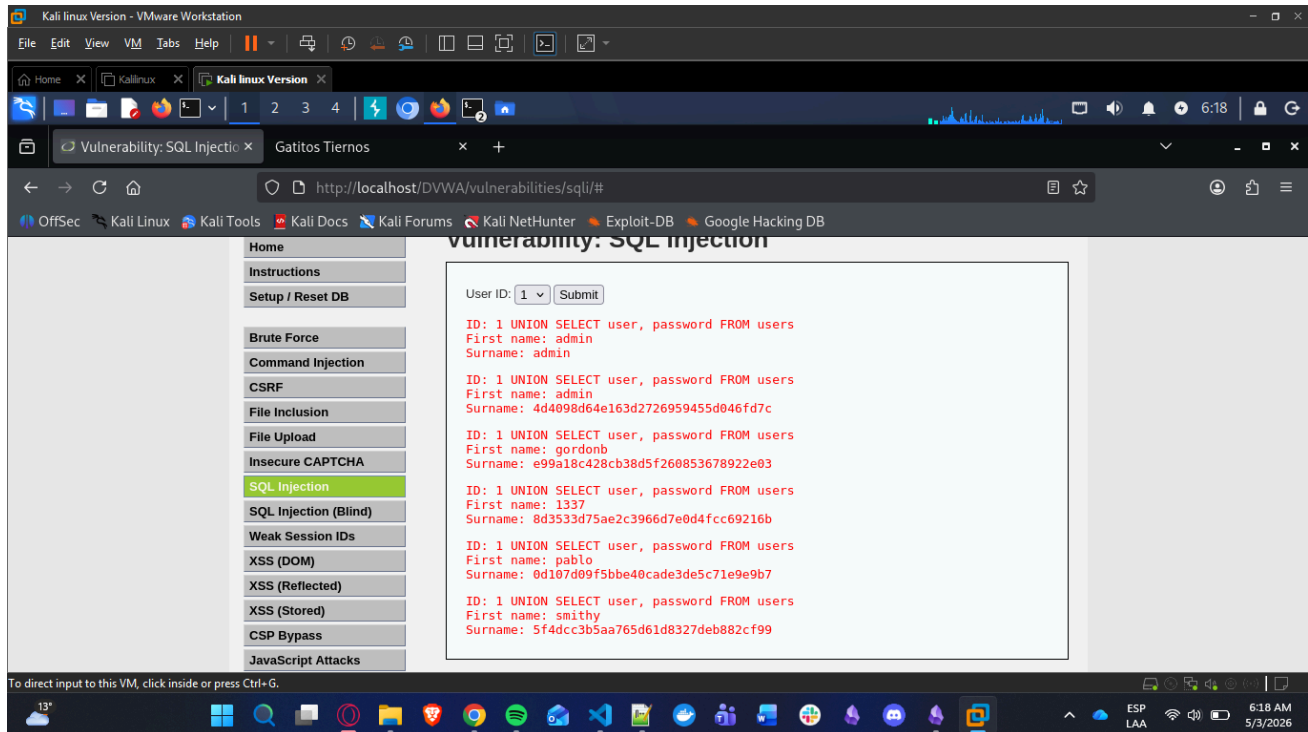


7. PHASE 2: EXPLOITATION OF INJECTION VULNERABILITIES

7.1 SQL Injection (SQLi)

- ◆ **Steps followed:** The Medium security level replaces the text input with a drop-down menu to restrict user input. We bypassed this UI restriction by capturing the `POST` request and directly modifying the `id` parameter. We injected a `UNION SELECT` statement to combine the legitimate query with our own query targeting the `users` table.
- ◆ **Tools and commands used:** Burp Suite (Proxy Intercept). Payload: `id=1 OR 1=1 UNION SELECT user, password FROM users`
- ◆ **Explanation of why the vulnerability exists:** The PHP script fails to enclose the integer `$id` variable in quotes within the SQL statement. Consequently, the `mysql_real_escape_string()` function is rendered useless, allowing our injected `OR 1=1` logic to execute as part of the SQL command structure.
- ◆ **Mitigation recommendation:** Implement Parameterized Queries (Prepared Statements) exclusively. Do not concatenate user input directly into SQL command strings.

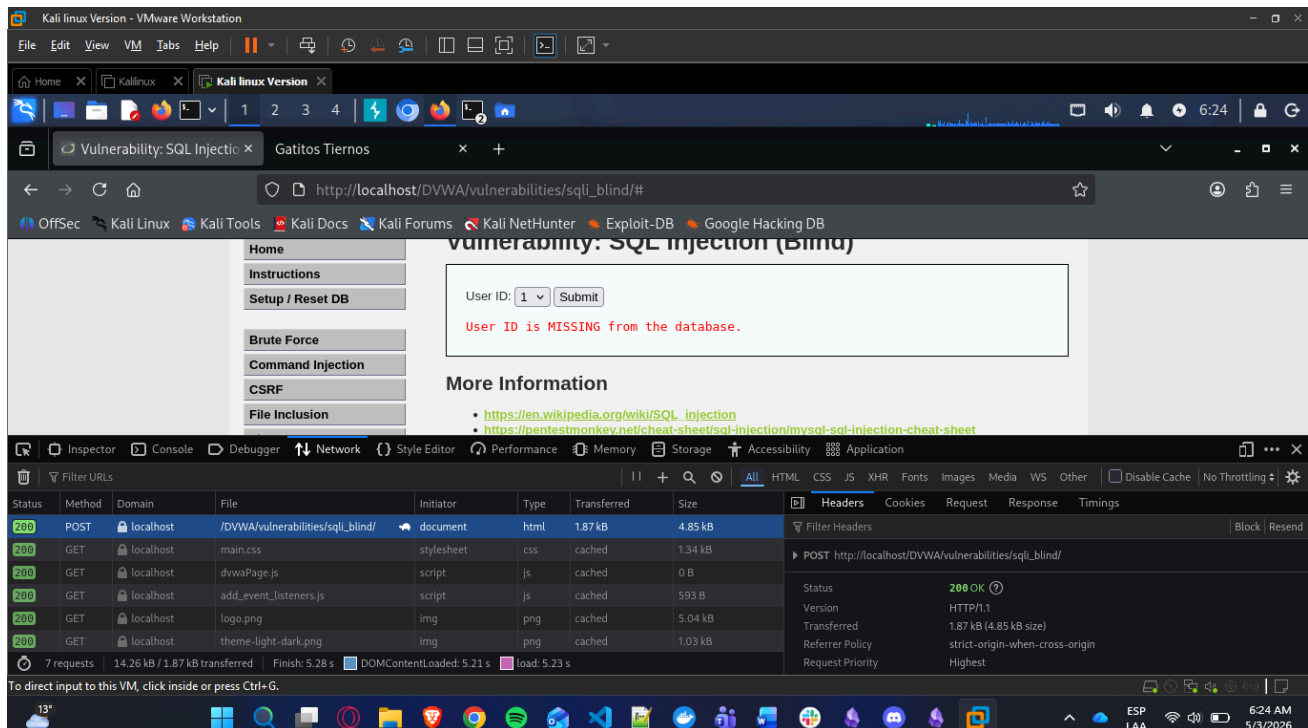
◆ Screenshot:



7.2 SQL Injection (Blind)

- ◆ **Steps followed:** Similar to standard SQLi, we bypassed the drop-down menu using Burp Suite. Because the application does not print database errors or data directly to the screen, we injected a time-based payload. We observed the server's response time to confirm the database executed our command.
- ◆ **Tools and commands used:** Burp Suite (Proxy and Repeater). Payload: `id=1 AND sleep(5)`
- ◆ **Explanation of why the vulnerability exists:** Identical to standard SQLi; the lack of quotation marks around the user-supplied integer allows direct manipulation of the query logic.
- ◆ **Mitigation recommendation:** Enforce Parameterized Queries (Prepared Statements) to ensure input is always treated as literal data, not executable code.

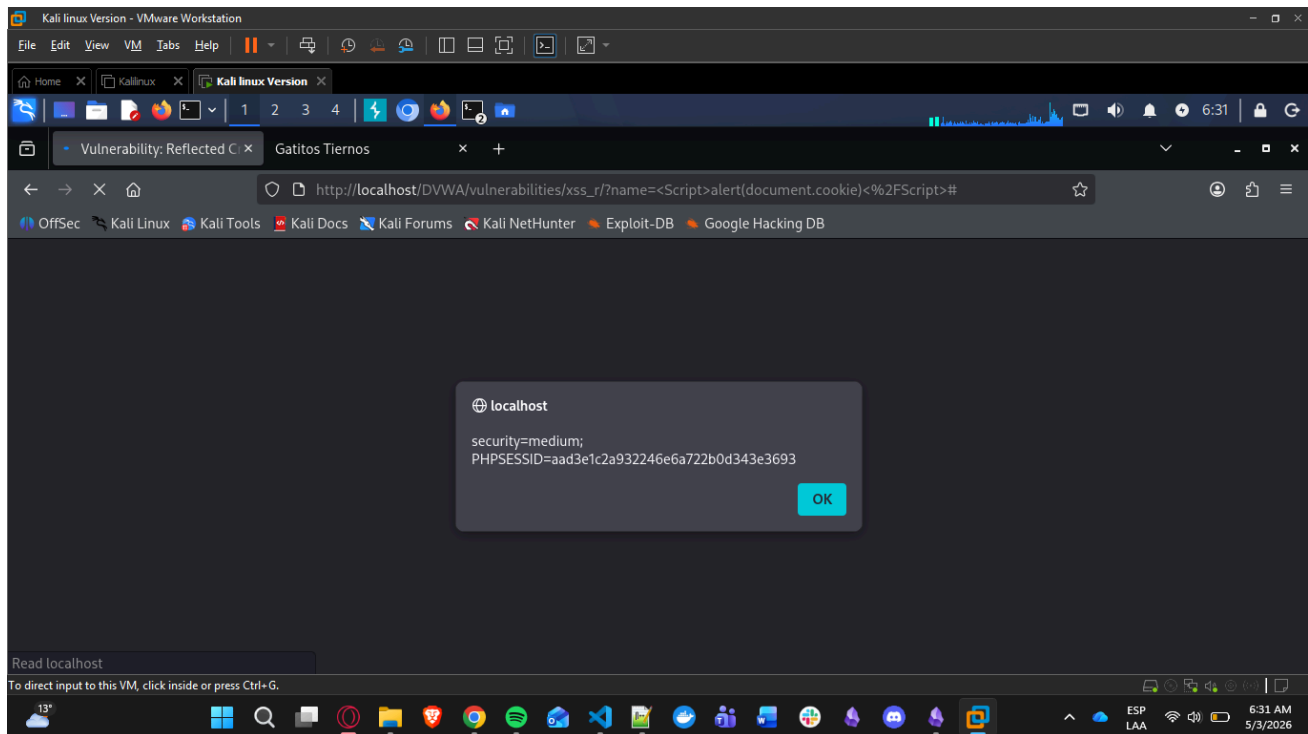
◆ Screenshot:



7.3 Cross-Site Scripting (Reflected)

- ◆ **Steps followed:** We inputted a malicious JavaScript payload into the 'name' field. Knowing the server filters the exact string `<script>`, we altered the capitalization of the first letter to bypass the filter and achieve execution in the browser.
- ◆ **Tools and commands used:** Firefox Web Browser. Payload: `?name=<Script>alert(document.cookie)</Script>`
- ◆ **Explanation of why the vulnerability exists:** The application uses a case-sensitive blacklist (`str_replace`) which only removes the specific lowercase string `<script>`.
- ◆ **Mitigation recommendation:** Use `htmlspecialchars()` to properly encode HTML entities before reflecting user input back to the browser. Avoid blacklists entirely.

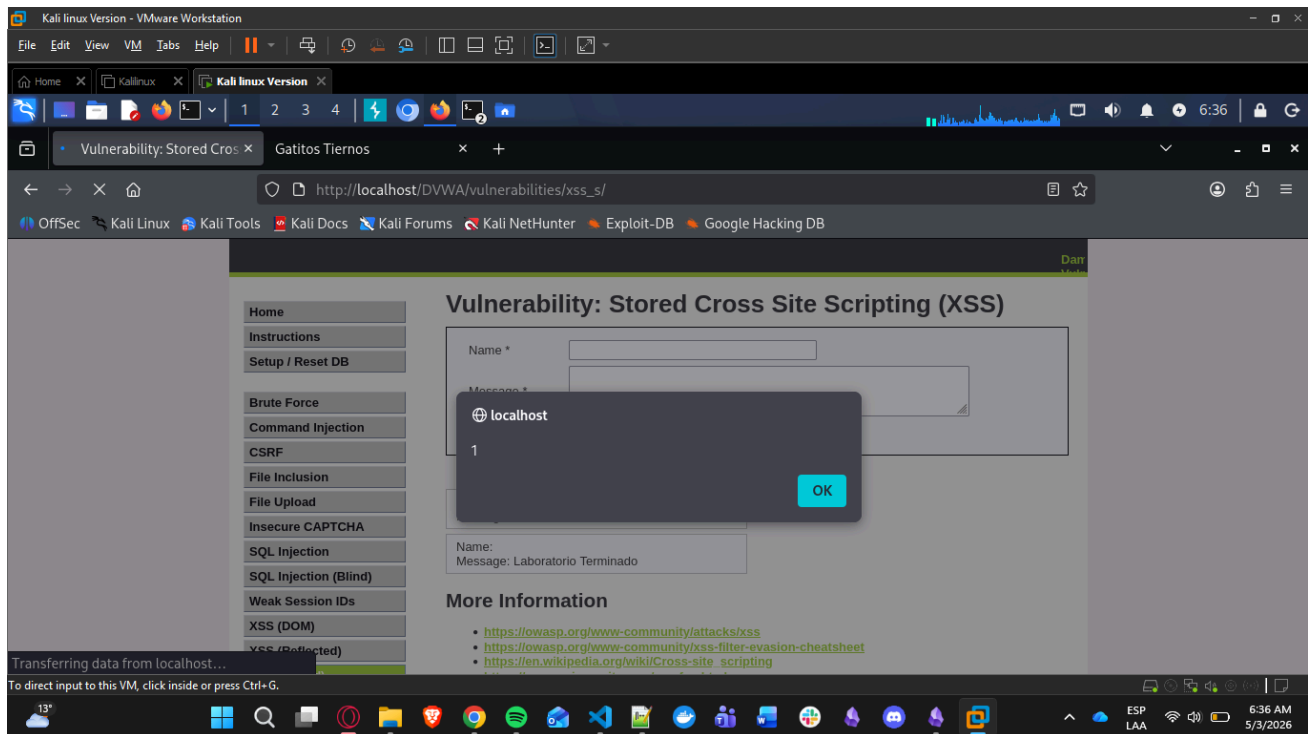
◆ Screenshot:



7.4 Cross-Site Scripting (Stored)

- ◆ **Steps followed:** The Guestbook module restricts the 'Name' input field using an HTML `maxlength` attribute. We filled out the form, intercepted the POST request, and replaced the name parameter with a payload using an image tag. This bypassed both the browser length restriction and the server's weak `<script>` blacklist.
- ◆ **Tools and commands used:** Burp Suite (Proxy Intercept). Payload: `txtName=<img src=x onerror=alert(1)>&mtxMessage=Test`
- ◆ **Explanation of why the vulnerability exists:** The application relies on client-side controls (`maxlength`) which are easily bypassed with a proxy. Furthermore, it sanitizes the database input using the same flawed blacklist as the reflected vulnerability.
- ◆ **Mitigation recommendation:** Validate input length on the server side. Apply context-aware output encoding (e.g., `htmlspecialchars()`) to all user inputs before rendering them on the page.

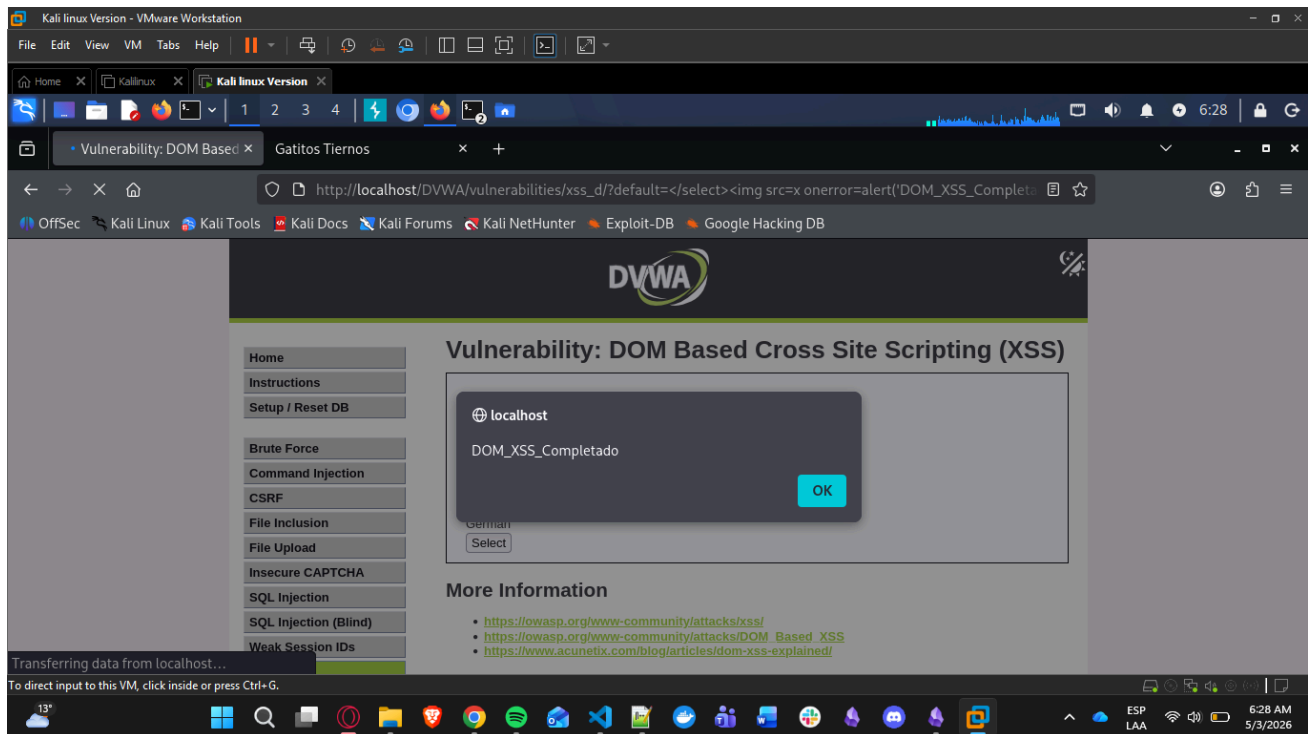
◆ Screenshot:



7.5 Cross-Site Scripting (DOM)

- ◆ **Steps followed:** We manipulated the `default` parameter in the URL. To bypass the server's `stripops` check for `<script>`, we closed the current HTML tag context and utilized an alternative execution vector (an image tag with an `onerror` handler).
- ◆ **Tools and commands used:** Firefox Web Browser. Payload: `?default=</select><img src=x onerror=alert('DOM')>`
- ◆ **Explanation of why the vulnerability exists:** The application unsafely writes the `default` URL parameter directly into the Document Object Model (DOM) via JavaScript. The server's attempt to filter the parameter is ineffective against alternative payload structures.
- ◆ **Mitigation recommendation:** Avoid writing unvalidated data directly to the DOM using sinks like `innerHTML` or `document.write`. Use safe properties like `textContent` instead.

◆ Screenshot:

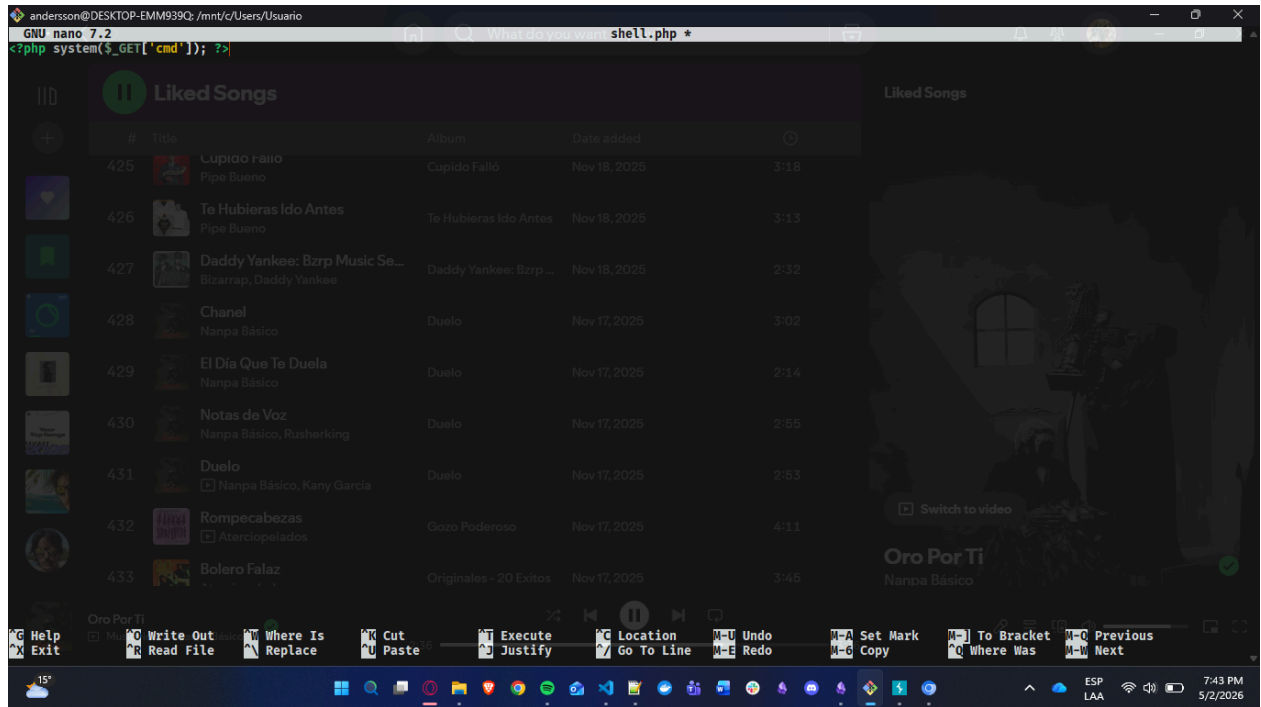


8. PHASE 3: ADVANCED EXPLOITATION AND SESSIONS

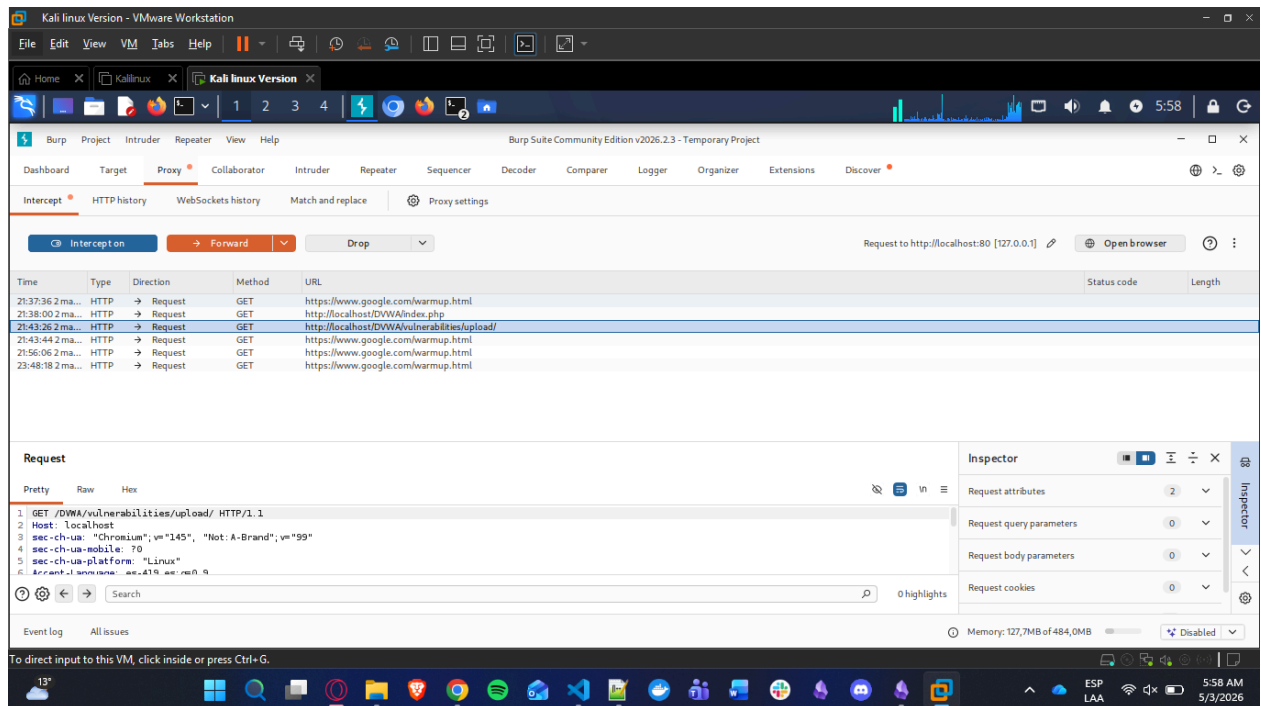
8.1 File Upload

- ◆ **Steps followed:** We created a malicious PHP web shell (`shell.php`). When uploading it, we intercepted the HTTP request in Burp Suite and manually altered the `Content-Type` header to make the server believe it was receiving an image.
- ◆ **Tools and commands used:** Text editor (to create shell), Burp Suite (Proxy Intercept). Command: Changed `Content-Type: application/x-php` to `Content-Type: image/jpeg`.
- ◆ **Explanation of why the vulnerability exists:** The server's validation logic entirely trusts the `Content-Type` header sent by the client. Because the client has full control over HTTP headers, this value can be spoofed trivially.
- ◆ **Mitigation recommendation:** Never trust client-supplied headers. Validate the file extension against a strict whitelist, verify the file's "magic bytes" (file signature) server-side, and rename the file upon storage outside the web root.
- ◆ **Screenshots:**

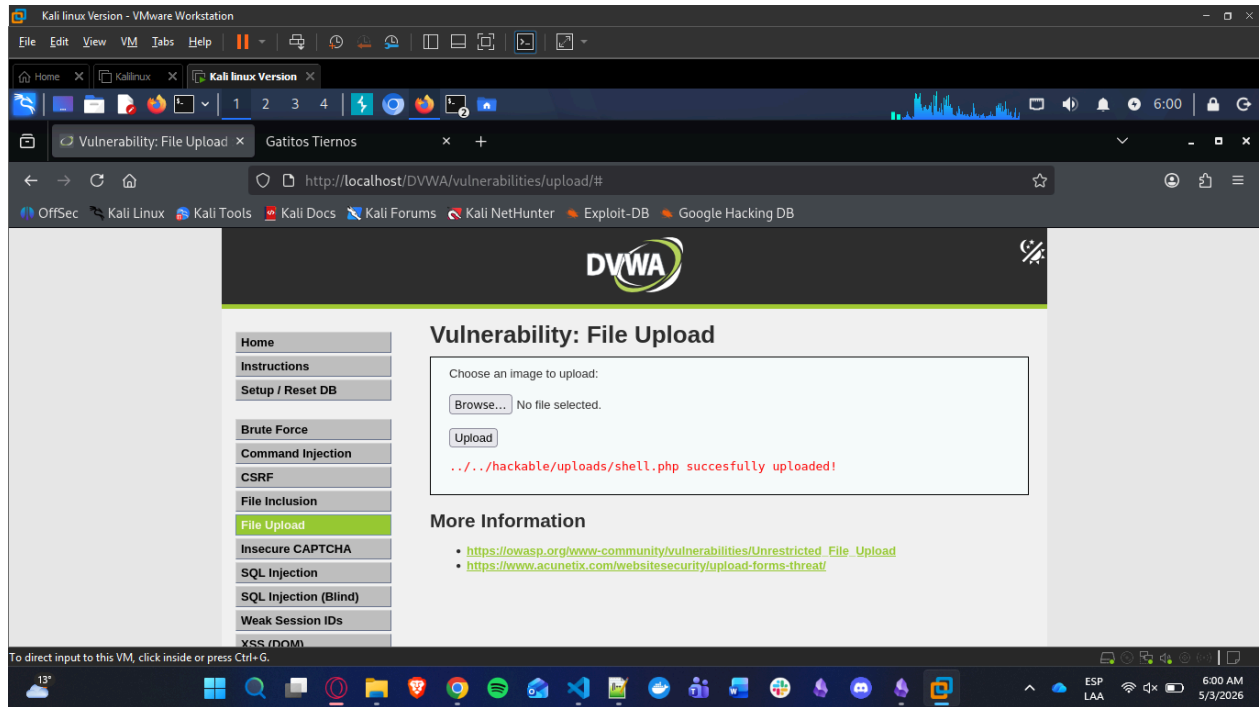
◆ Shell Creation:



◆ Burp Intercept:



◆ Success:

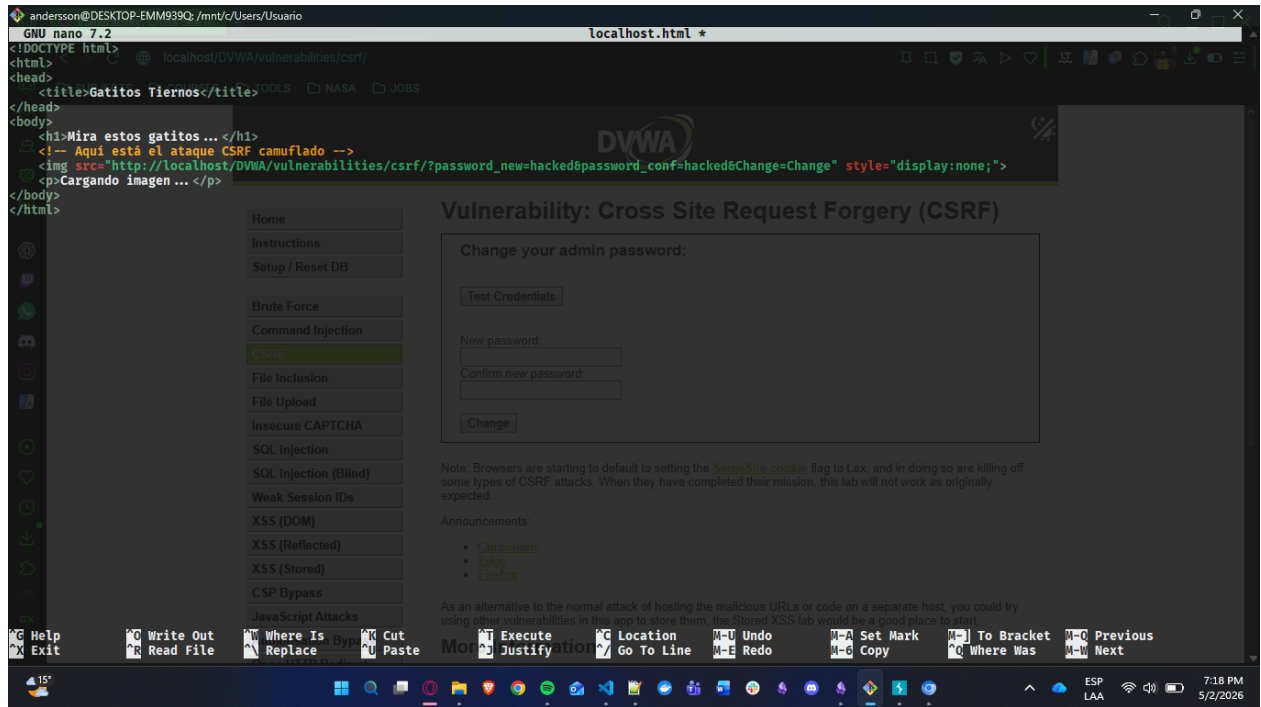


8.2 File Inclusion (LFI)

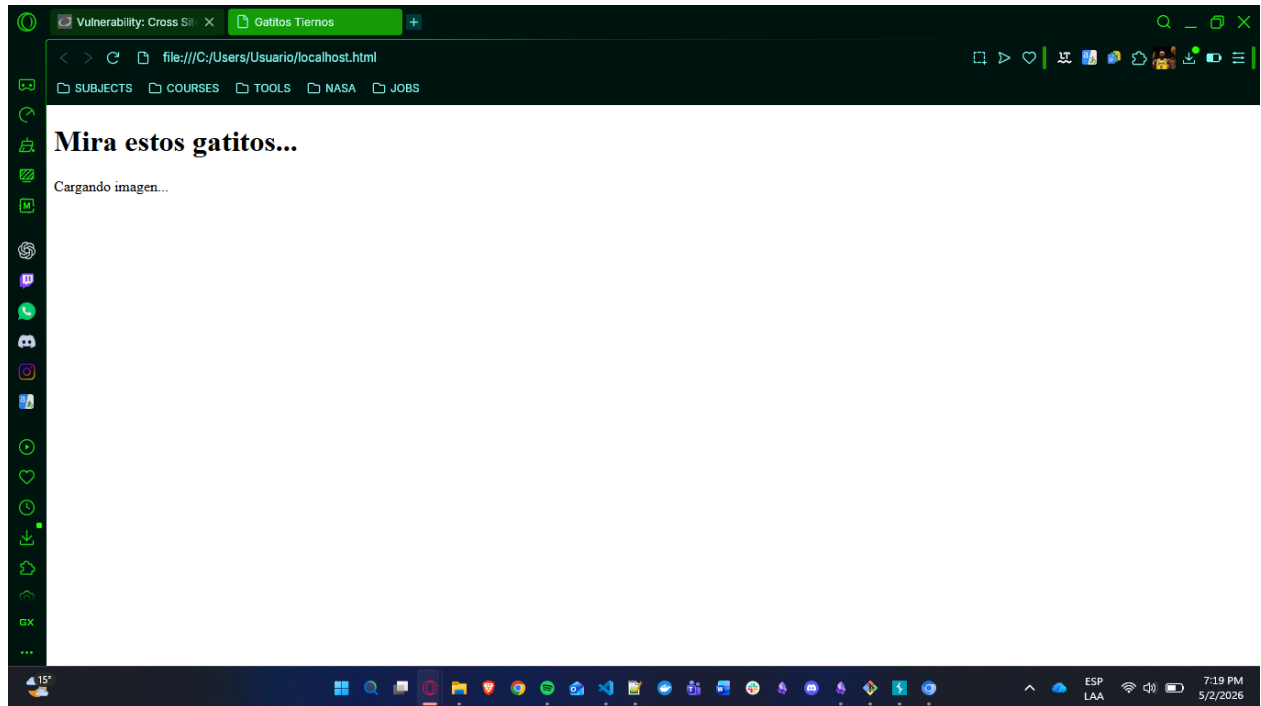
- ◆ **Steps followed:** To read the server's password file, we targeted the `page` parameter. Knowing the server filters standard traversal sequences (`../`), we utilized nested sequences to reconstruct the path after the filter executed.
- ◆ **Tools and commands used:** Firefox Web Browser. Payload: `?page= ... /./ ... /./ ... /./etc/passwd`
- ◆ **Explanation of why the vulnerability exists:** The developer utilized `str_replace()` to remove `../` strings, but this function is not recursive. By injecting `... /./` , the inner `../` is removed, leaving the outer characters to seamlessly form a functional `../` string.
- ◆ **Mitigation recommendation:** Do not pass user input directly into filesystem APIs. Use an indirect object reference (e.g., mapping `page=1` to `about.php` server-side) or rigorously validate input against a strict whitelist.

- ◆ **Steps followed:** We crafted a malicious HTML file containing an auto-submitting form targeting the DVWA password change URL. To bypass the Medium security `HTTP_REFERER` check, we specifically named our file `localhost.html` and disguised it behind a link offering cute cats.
- ◆ **Tools and commands used:** HTML/Text Editor, Firefox Web Browser. Payload: Malicious HTML form pointing to `http://localhost/DVWA/vulnerabilities/csrf/`.
- ◆ **Explanation of why the vulnerability exists:** The application attempts to verify trust using `stripos( $_SERVER[ 'HTTP_REFERER' ] ,$_SERVER[ 'SERVER_NAME' ] )`. `stripos` only checks if the `SERVER_NAME` ("localhost") exists *anywhere* within the Referer header, allowing an attacker to satisfy this weak check by controlling the filename or directory name of their exploit page.
- ◆ **Mitigation recommendation:** Implement unpredictable, session-specific Anti-CSRF tokens that are validated server-side on every state-changing request. Do not rely on `HTTP_REFERER` for authorization.
- ◆ **Screenshots:**

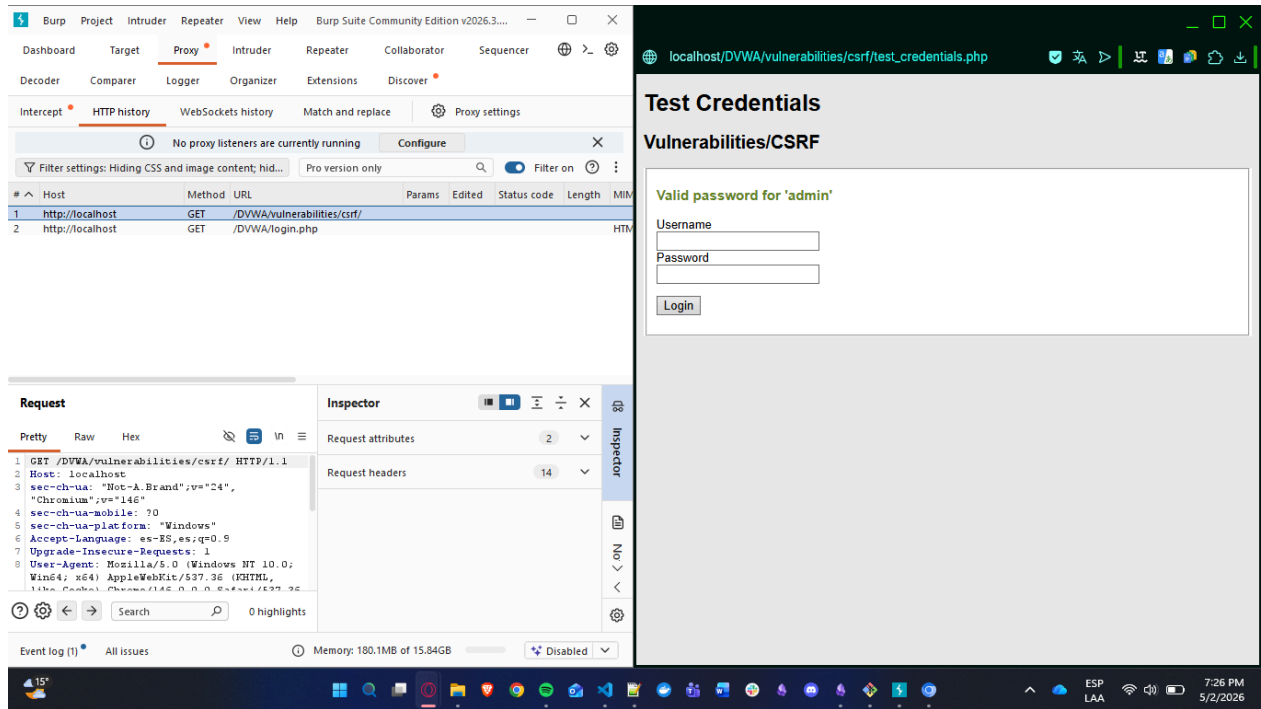
◆ Payload File:



◆ Bait Page:



◆ Success:



9. TECHNICAL DISCUSSION: BYPASSING DEFENSES

? Which vulnerability was hardest to exploit at medium difficulty, and what additional defense mechanism was DVWA using? How did you identify and bypass it?

Answer: The **Cross-Site Request Forgery (CSRF)** vulnerability was the most complex to exploit at the Medium level because it required setting up an external mechanism to trick the server's contextual logic.

- ◆ **Defense Mechanism:** DVWA implemented an `HTTP_REFERER` check, an additional layer verifying that the request to change the password seemingly originated from the DVWA server itself (`localhost`).
- ◆ **Identification:** By reading the server-side code (`stripes( $_SERVER[ 'HTTP_REFERER' ], $_SERVER[ 'SERVER_NAME' ]) !== false`), we identified the specific flaw: it only checked if the string "localhost" was present *anywhere* in the referer URL, not necessarily as the actual host origin.
- ◆ **Bypass:** We bypassed this defense by creating a standalone, malicious HTML file and deliberately naming it `localhost.html` . When the victim accesses this file, their browser

automatically appends the filename to the Referer header. This tricked the `stripes` function into returning `true`, executing our forged password change request.



10. FINAL CONCLUSIONS

Based on the simulated attacks against the DVWA environment, we conclude the following:

1. **The Fallacy of Blacklisting:** Throughout the lab, the Medium difficulty relied heavily on blacklists (blocking `<script>`, `../`, or checking `HTTP_REFERER`). Blacklists are fundamentally flawed because attackers will always find alternate encodings or nested patterns to bypass them.
2. **Client-Side Trust is Fatal:** Relying on HTTP headers (like `Content-Type`) or hidden HTML form elements for security is ineffective, as an attacker utilizing a proxy has absolute control over this data before it reaches the server.
3. **Universal Mitigation:** The organization must completely overhaul its input validation strategies.
 - ◆ **For SQLi:** Implement Parameterized Queries exclusively.
 - ◆ **For XSS:** Enforce strict Context-Aware Output Encoding and implement a Content Security Policy (CSP).
 - ◆ **For CSRF:** Implement unpredictable, session-specific Anti-CSRF tokens validated server-side on every state-changing request.
 - ◆ **For Uploads/LFI:** Store uploads outside the web root, rename them, and use indirect object references (mapping IDs to files) rather than accepting direct file paths.



11. BIBLIOGRAPHIC REFERENCES

- ◆ **OWASP Foundation.** (2026). *OWASP Top 10:2021 The Most Critical Web Application Security Risks*.
- ◆ **PortSwigger.** (2026). *Web Security Academy: SQLi, XSS, File Upload, and CSRF Documentation*.
- ◆ **Stuttard, D., & Pinto, M.** (2026). *The Web Application Hacker's Handbook*.
- ◆ **DVWA Documentation.** (2026). *Damn Vulnerable Web App Source Code Analysis Guide*.

